McGill University
School of Computer Science

Training and Evaluation Practices

Assignment 2 Report

Prof. Pascale Gourdeau
COMP 551 - Applied Machine Learning

Steven Chang	261239062
Mujtaba Mirhasan	261213939
Matthew Lacasse	261183472

27 February 2026

***Abstract*—This project investigates spam classification using logistic regression on the UCI Spambase dataset, which contains 57 continuous features describing word, special characters, and capitalization patterns.**

In Task 1, logistic regression was implemented from scratch and optimized using Stochastic Gradient Descent (SGD). The effects of learning rate, batch size, epochs, and L2 regularization were analyzed through training curves. They highlight the trade-offs between convergence speed, stability, and overfitting.

Task 2 applied 3-fold cross-validation to tune hyper-parameters using mean cross-entropy as the primary evaluation metric. The optimal configuration, e.g. $\eta = 0.01$, the batch size $|\mathbb{B}| = 1$, epochs = 50, achieved strong performance with low variance across folds, demonstrating stable generalization despite the training split of only 5% of the whole dataset.

In Task 3, an L2 regularization sweep was conducted to study the bias–variance trade-off. Results confirmed that small regularization increased variance, while large regularization increased bias. Our variance was very small, indicating underfitting. The optimal lambda was $\lambda = 0.0$, however $\lambda = 10^{-5}$ was chosen for numerical stability.

Finally, Task 4 used L1-regularized logistic regression to study sparsity and feature importance. As regularization increased, many coefficients shrank to zero, leaving key features—such as exclamation marks, brackets, and words like "our" and "credit"—active. About 30 features were removed without significant accuracy loss, showing the model can be interpretable.

I. Introduction

This project focuses on classifying e-mails as spam or non-spam using logistic regression. The dataset used is the UCI Spambase dataset, which contains 57 continuous input features and a binary target variable indicating whether an e-mail is spam. The logistic model that will be trained should be able to classify these features into a binary classification.

The first 48 features capture the frequency of specific words in the e-mail. The next 6 represent the frequency of certain special characters. The last 3 describe capital letter usage: the average length of uninterrupted sequences, the length of the longest sequence, and the total number of capital letters. These features reflect common spam patterns that can be used to determine if an email is spam or not.

The model's weights are optimized via Stochastic Gradient Descent (SGD), which iteratively updates weights based on batch processing. Considering only 5% of the dataset is allocated for training, the model is highly susceptible to overfitting; consequently, L2 and L1 regularization techniques are used to penalize large weights, which can reduce the effects of overfitting.

Tasks 2 and 3 tune the hyper-parameters of the model (learning rate, batch size, epochs) and explore the bias–variance trade-off via L2 regularization using K-fold cross-validation. Manipulating these hyper-parameters shows the impact of each on the final model, the loss in general, and convergence speed.

Task 4 applies L1-regularized logistic regression via scikit-learn. L1 promotes sparsity by setting some coefficients to zero, which lets us know which features are less important in our model. A regularization path is constructed by varying the inverse regularization parameter C, showing how sparsity, feature selection, and performance change with regularization.

II. Task 1: Logistic Regression with SGD

A. Preprocessing

The dataset was first loaded as numerical values and separated into input features X and binary target label y. To ensure randomness, the data was shuffled using a seed (the value of the seed is 2026) for reproducibility. A split was then performed on the dataset, allocating 5% of it for training and 95% for testing.

After that, feature standardization was applied to improve optimization stability during SGD. The mean and the standard deviation of each feature were computed using only the training set in order to prevent data leakage. If they were computed using both the training set and the test set, the model would indirectly obtain information about the test distribution before evaluation. The same parameters (training mean and standard deviation) were subsequently applied to the test set for consistency.

B. Model and Training

A binary logistic regression model is used to estimate the probability that an e-mail is spam. Given

input features $x \in \mathbb{R}^d$, where d represents the amount of features, the model computes:

$$p(y = 1 \mid x) = \sigma(w^\top x + b),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid function, $w \in \mathbb{R}^d$ is the weight vector, z is the computed logit of the observed features with the weights, and b is the bias term. Since the classification is binary, the probability of the class being 0 is 1 minus the probability of the class being 1. Stochastic Gradient Descent (SGD) utilizes batches of entries in our dataset, therefore, each batch is defined like the following:

$$D = \{(x^{(n)}, y^{(n)})\}_{n=1}^{N}$$

$$D_{\mathbb{B}} = \{(x^{(i)}, y^{(i)}) \mid i \in \mathbb{B}\}$$

$$\mathbb{B} \subset \{1, \ldots, N\}, \qquad |\mathbb{B}| = \text{batch_size}$$

where $\mathbb{B}$ represents the set of indices selected for the current update. The objective function is the average binary cross-entropy loss with L2 regularization on a randomly selected batch $\mathbb{B}$:

$$\begin{aligned}\mathcal{L}_{\mathbb{B}}(w, b) = &- \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} [y_i \log(\hat{y}_i) \\ &+ (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|w\|_2^2\end{aligned} \tag{1}$$

Gradients are computed with respect to the convention $\lambda \|w\|^2$, yielding:

$$\nabla_w \mathcal{L}_{\mathbb{B}} = \frac{1}{|\mathbb{B}|} X^\top (\hat{y} - y) + 2\lambda w \tag{2}$$

$$\nabla_b \mathcal{L}_{\mathbb{B}} = \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} (\hat{y}_i - y_i) \tag{3}$$

Typically, the bias term should not be affected by this regularization process, therefore, as indicated above, the gradient of the bias term differs from the other weights. The Optimization is performed using stochastic gradient descent (SGD) with mini-batches. At each epoch, the training data is shuffled and split into batches of size $|\mathbb{B}|$. The parameters are updated using a fixed learning rate η for a specified number of epochs. To analyze convergence behavior, different learning rates, batch sizes, and regularization strengths are explore.

C. Results and Discussion

Training curves were generated by recording the regularized cross-entropy loss at the end of each epoch. The plots compare different learning rates ($\eta \in \{0.1, 0.01\}$), batch sizes ($|\mathbb{B}| \in \{1, 16\}$), numbers of epochs (50 and 120), and regularization strengths ($\lambda \in \{0, 10^{-3}\}$).

In general, larger learning rates (e.g., $\eta = 0.1$) converge more rapidly, but may exhibit higher oscillations, particularly with smaller batch sizes. Smaller learning rates (e.g., $\eta = 0.01$) produce smoother convergence but require more epochs to reach similar loss levels.

Mini-batch size also influences training behavior. A batch size of 1 ($|\mathbb{B}| = 1$) introduces higher variance in gradient updates: the top graphs in Figures A.1a and A.1b have noisier training curves. Larger batch sizes ($|\mathbb{B}| = 16$) provide more stable gradient estimates and smoother loss curves, which helps prevent oscillations.

When L2 regularization is applied, the training loss decreases more slowly and stabilizes at a slightly higher value, reflecting the penalty imposed on large weights. This can be seen when comparing every graph from Figure A.1a to its corresponding graph in Figure A.1b. This happens because the model without regularization overfits the data, making the loss very small. This behavior is consistent with the bias–variance trade-off and motivates further analysis of regularization effects in subsequent tasks.

III. Task 2: Hyper-parameter Tuning

A. K-Fold Setup

To evaluate the hyper-parameter configurations, K-fold cross-validation was performed, with a $K = 3$. The dataset was shuffled before splitting to ensure each fold is representative. For each hyper-parameter configuration, the model is trained on $K - 1$ folds and evaluated on the held-out fold, repeating this for all folds. The metrics collected were mean binary cross-entropy (loss), mean accuracy, and standard deviation across folds to assess stability.

The hyper-parameters explored were:

Hyper-parameter	Search Space
Learning rate (η)	{1, 0.1, 0.01, 0.001, 0.0001}
Batch size ($\|\mathbb{B}\|$)	{1, 16, 64}
Number of epochs	{50, 100, 150, 200}
L2 regularization (λ)	10^{-3}

TABLE III.1: Hyper-parameter Grid Search Space

B. Results and Reflection

The grid search produced the following summary of cross-validation results (Table 3.1). For each configuration, we report mean cross-entropy, mean accuracy, and standard deviation across folds.

η	$\|\mathbb{B}\|$	Epoch	Mean L.	Std L.	Mean Av.	Std Av.
0.01	1	50	0.220	0.010	0.900	0.032
0.1	64	200	0.221	0.011	0.909	0.038
0.1	16	100	0.221	0.008	0.900	0.032

TABLE III.2: Subset of results; full table: Appendix A.

Based on these results, the configuration that minimized mean cross-entropy was chosen as the best. The hyper-parameters for the model are: $\eta = 0.01$, $|\mathbb{B}| = 1$, and epochs $= 50$. Although other configurations achieved slightly higher mean accuracy, cross-entropy is the primary metric for logistic regression, so this configuration is optimal. The standard deviation across folds is low, indicating that the performance of the models did not fluctuate significantly regardless of which subset of data the model was trained on.

The models which had the highest mean loss had the following hyper-parameters were all the ones with $\eta = 1$ and a batch side of $|\mathbb{B}| = 1$. The combination of a high learning rate with small batch size can create very strong oscillations and the weights may not converge properly. The models which had the second highest mean loss were the ones with too little of a learning rate (e.g. $\eta = 0.0001$).

Among the hyper-parameters, learning rate and batch size were the most influential. The learning rate controls the step size during gradient updates: too high caused unstable convergence, too low slowed training. The batch size affected the gradient noise: smaller batches introduced stochasticity, which helped generalization in this case. Finally, The number of epochs mattered less because once the model converged, 50 epochs were sufficient with this learning rate and batch size.

In the light of these observations, we have modified the search space for each hyper-parameters and decided to re-run the experiment with parameters we think will perform well to fully capture the high potential of our model. The learning rate and batch size were modified with additional values, as they are the most influential, and the experiment was done again:

Hyper-parameter	Search Space
Learning rate (η)	$\{1, 10^{-1}, 5\times10^{-2}, 2\times10^{-2}, 10^{-2}, 10^{-3}, 10^{-4}\}$
Batch size ($\|\mathbb{B}\|$)	{1, 16, 32, 64, 128}
Number of epochs	{50, 100, 150, 200}
L2 reg. (λ)	10^{-3}

TABLE III.3: Hyper-parameter Grid Search Space.

η	$\|\mathbb{B}\|$	Epoch	Mean L.	Std L.	Mean Av.	Std Av.
0.05	16	150	0.220	0.010	0.900	0.032
0.10	32	150	0.221	0.010	0.900	0.032
0.01	1	50	0.221	0.010	0.900	0.032

TABLE III.4: Top-performing results with additional parameters.

From Table III.4, we observe that adding onto the initial search space does not significantly find any better combinations for the mean loss. The top performer from the initial search is still present in the top performers. The ones at the top contain higher batch sizes, which are more expensive, and higher epochs for a slight mean loss improvement. These slight gains do not justify the increased computational cost, since the model has essentially reached a convergence plateau.

With these observations, we have decided to pick the model with $\eta = 0.01$, the batch size $|\mathbb{B}| = 1$, epochs = 50. This is the best trade-off for computing efficiency and accuracy.

IV. Task 3: Bias–Variance Trade-Off via L2

A. Training Hyper-parameters

For this section of the report, we utilize fixed hyper-parameters and search through a λ grid to find which how the parameter λ affects the bias-variance results of our model. Like previously, we have performed this search using K-fold cross-validation, with $K = 3$. The dataset was shuffled before splitting. The following hyper-parameters were set as fixed:

As for the regularization coefficient λ, the following values were studied:

Hyper-parameter	Value
Learning rate (η)	0.1
Batch size ($\|\mathbb{B}\|$)	16
Number of epochs	200

TABLE IV.1: Fixed Hyper-parameters

Hyper-parameter	Search Space
Regularization Coefficient (λ)	$\{0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$

TABLE IV.2: Hyper-parameter Grid Search Space.

B. Cross-Validation Results

As seen from the generated plots A.2b, as the regularization coefficient λ decreases, we observe a higher model variance between the train set and the validation set. As the coefficient grows, this variance lowers (the difference between the loss of the train and validation sets becomes smaller), however the bias grows (both curves grow higher). Similarly, on the accuracy graph, as lambda increases, we see an immediate drop in accuracy as soon as it reaches a certain threshold.

C. Discussion

Lower values of λ do not impose high penalty on the weight sizes, in other words, a low λ still penalizes high weights, but some high weights may be capable of achieving a lower cost despite the penalty on their size. When the coefficient grows, this limits how big the weights can be, and forces them to shrink to lower the cost, which may yield an inadequate solution. A bigger lambda penalizes the weight sizes more, and some solutions that achieved a lower loss with big weights are no longer feasible. This is expected.

The validation curve typically has a U-shape. Too low of a λ indicates high variance, and too high of a λ indicates high bias. The middle of this U-shape indicates the right balance between the variance and bias of our model. In our case, we do not observe this U-shape, because the variance in our model is too little compared to the bias for the generalization error. The distance between the train loss and the validation loss is approximately 0.13 units, whereas the bias towards a low regularization coefficient can reach up to 0.6. As soon as the variance drops, the bias grows faster.

The best lambda given by our code is $\lambda = 0.0$; however, lambdas from 10^{-9} to 10^{-5} do not significantly affect the validation loss. The main issue with our algorithm comes from underfitting, and not overfitting. Setting $\lambda = 0.0$ is typically not recommended for numerical stability; therefore, we decided to opt for a low lambda, but not high enough to cause more validation loss: $\lambda = 10^{-5}$. Testing first with $\lambda = 0.0$, we obtain a test cross-entropy cost of $J(w) = 0.201$, and with $\lambda = 10^{-5}$, the test cross-entropy is $J(w) = 0.202$. The difference in cost is $\epsilon = 5.15 \times 10^{-4}$ which is a very low error while keeping regularization. The decided lambda for this model is the following:

Hyper-parameter	Final Value
Learning rate (η)	0.1
Batch size ($\|\mathbb{B}\|$)	16
Number of epochs	200
Regularization Coefficient (λ)	10^{-5}

TABLE IV.3: Final Model Configuration for Task 3 Evaluation

With the final cross-entropy cost being 0.202, and the test accuracy being equal to 89.4%.

V. Task 4: L1-Regularized Logistic Regression

A. Methodology

While L2 regularization is great at reducing the effects of overfitting, using L1 regularization is a great way to obtain sparse weights for the parameters of our model. As we increase λ, our regularization coefficient, we should see certain weights converge to 0, which indicates to us the importance of each parameter in our model. The weights that converge to 0 the fastest as we grow λ are typically the least important weights in our model. In this section, we utilize the logistic regression model from scikit-learn, using the penalty mode "l1". The model from scikit-learn uses the regularization parameter C, which is equivalent to $\frac{1}{\lambda}$. The following table shows all the parameters applied to the model:

Hyper-parameter	Value
l1_ratio	1.0
solver	"saga"
max_iter	5000
random_state	2026
warm_start	True

TABLE V.1: Hyper-parameter Configuration for Feature Selection

To analyze the importance of each parameter, we will use a search-space for C and will plot the weight value as a function of C to essentially create a regularization path of the coefficients graph:

Hyper-parameter	Search Space
Regularization Coefficient (C)	range(-4, 4, 30)

TABLE V.2: Varying Regularization Coefficient

We will also plot the sparsity of the model, which will indicate the number of non-zero coefficients as a function of C, and plot the accuracy of our model with respect to C. Since the coefficient C is inversely proportional to λ, we expect the weights in the regularization path to converge to zero as C becomes smaller, and the number of non-zero coefficients to grow as C grows. As we decrease C, we expect our model to see a trend similar to our L2 regularization model, where bias increases as C decreases

B. Discussion

Feature Index	Weight Coefficient	Description
Feature 52	0.2137	Symbol "!"
Feature 51	0.1637	Symbol "["
Feature 6	0.0603	Word "our"
Feature 20	0.0551	Word "credit"
Feature 23	0.0013	Number "000"

TABLE V.3: Non-zero weight magnitudes at $C = 0.0304$ (Best features)

Feature Index	Weight Coefficient	Description
Feature 31	0	Word "telnet"
Feature 28	0	Number "650"

TABLE V.4: Zero weight magnitudes for additional features at $C = 32.9$ (Worst features)

Looking at table $V.3$, we can see that at very low C, the features 52, 51, 6, 20, 7 & 23 are the only weights that are non-zero. This indicates that these features tend to be more important than the others. Furthermore, feature 28 & 31 are the first eliminated features, which indicates these features are not very useful in predicting the class of an email.

The generated plots $A.3d$ show several interesting features of our model. Graph A shows the regularization path, and we observe certain weights live longer than others. As predicted, as C shrinks, we observe the weights slowly converge to zero, indicating that as the regularization strength increases, feature weights die out.

When looking at the sparsity versus regularization strength graph, we see that as C increases, we get more non-zero features, which matches the expected trend and the regularization path diagram. At $C > 10$, we observe almost no zero-valued weights.

The cross-validation performance versus the coefficient C indicates that our model is capable of predicting the class of email efficiently when $C > 2 \times 10^{-1}$. This indicates that the following features are not significantly useful for predicting the class:

Zeroed Features	0, 1, 2, 3, 5, 8, 9, 10, 12, 13, 14, 16, 27, 28, 29, 31, 32, 33, 35, 37, 39, 40, 42, 46, 47, 48, 49, 50, 54, 55
Feature Count	30 features removed

TABLE V.5: Features eliminated by L1 Regularization.

Finally, the model accuracy versus C shows the exact answer we derived above, where at around 27 features, we get a great accuracy regardless of 30 features being dropped.

VI. Conclusion

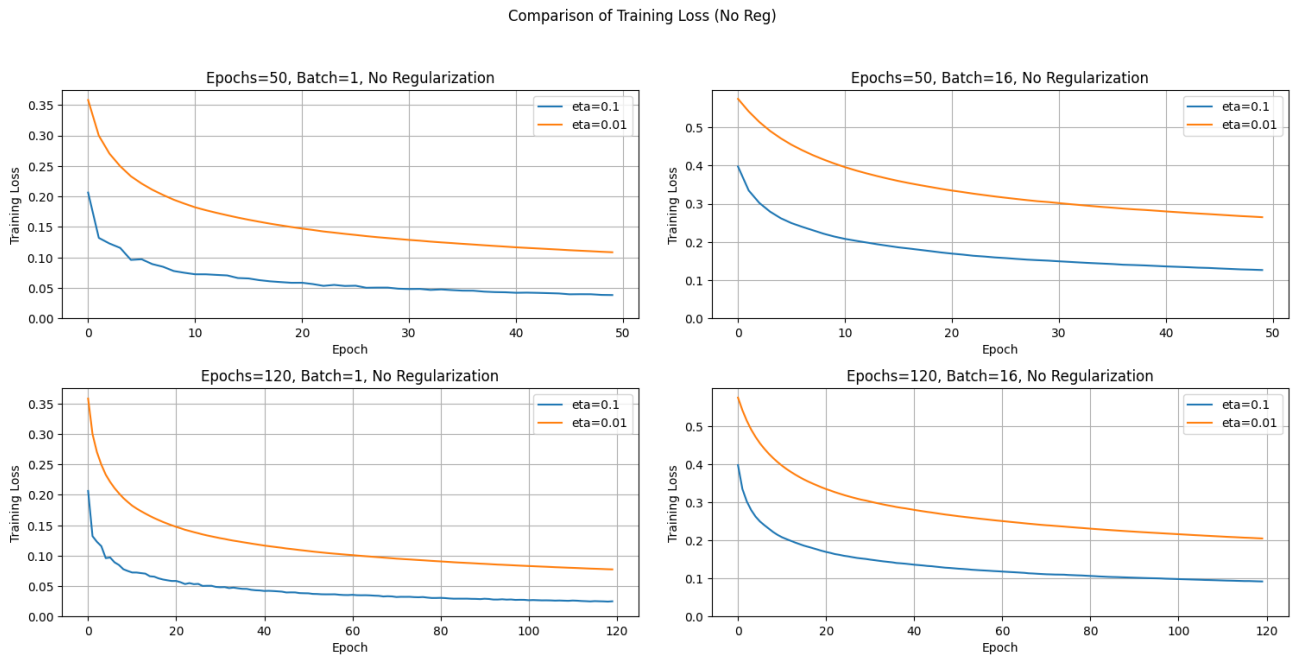
This project developed a logistic regression model for classifying e-mails as spam or non-spam, examining the effects of optimization, regularization, and feature selection. Hyper-parameter tuning of learning rate, batch size, and regularization strength was shown to influence convergence, stability, and generalization. L2 regularization managed the bias–variance trade-off, while L1 regularization produced a sparse, interpretable model by eliminating less informative features without significantly reducing accuracy. The findings highlight the importance of balancing model performance and having good computational efficiency in real-world classification tasks.

In addition, it would have been interesting to analyze the F1-score graph for different λ to evaluate performance, since dataset is imbalanced with 39.4% spam against 60.6% non-spam. It suggests that accuracy alone may not fully reflect predictive quality.

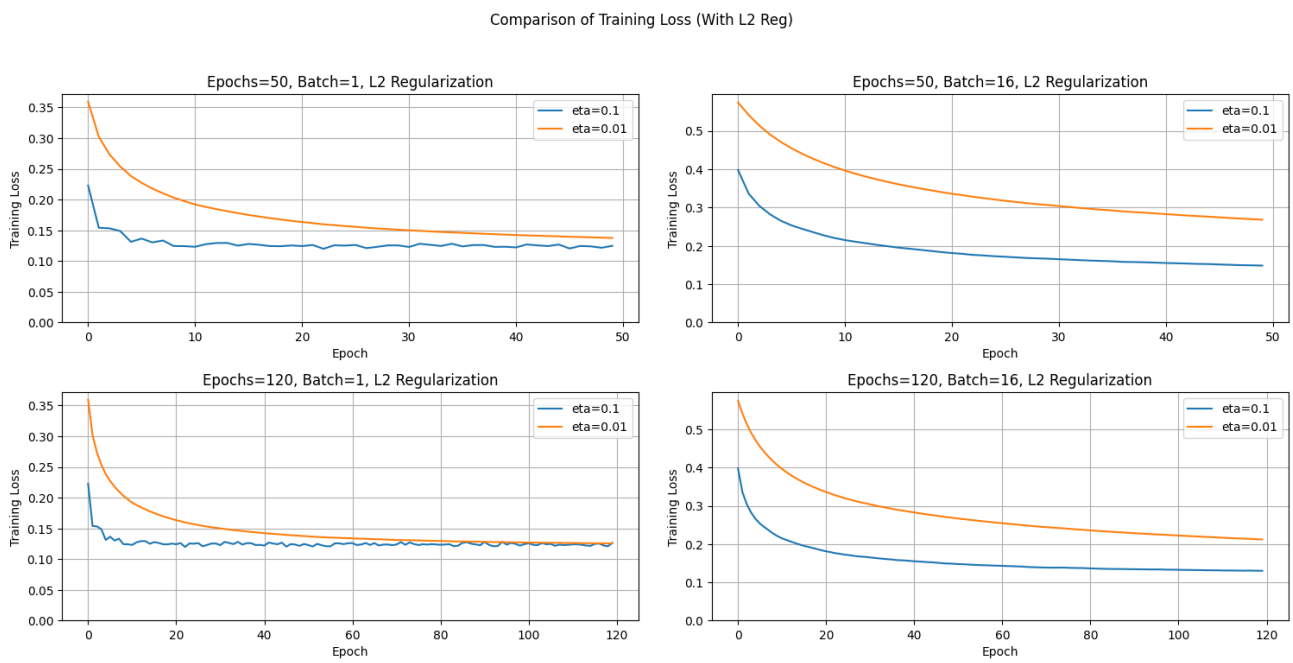
Statement of Contributions

Two (2) team members have contributed to the general structure of the code (Steven Chang, Mujtaba Mirhasan), while one (1) member has worked on the laboratory report and has contributed to various modifications in the code for additional analysis (Matthew Lacasse).

APPENDIX

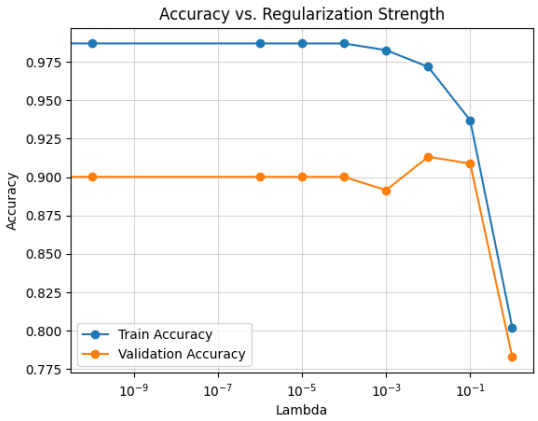


(a) Training loss curves without L2 regularization.

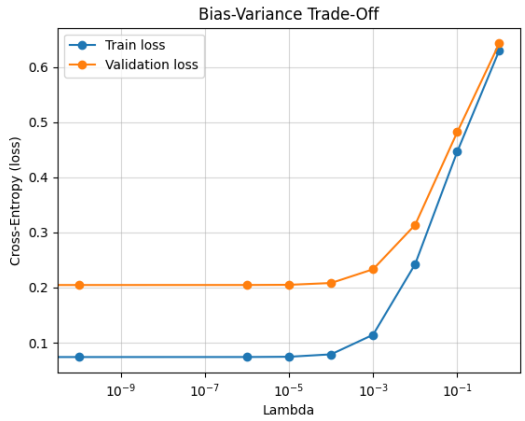


(b) Training loss curves with L2 regularization.

Fig. A.1: Training loss curves for the model, with and without L2 regularization.

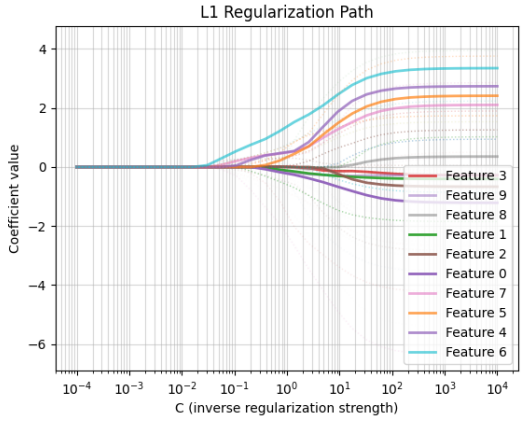


(a) Model accuracy with varying λ

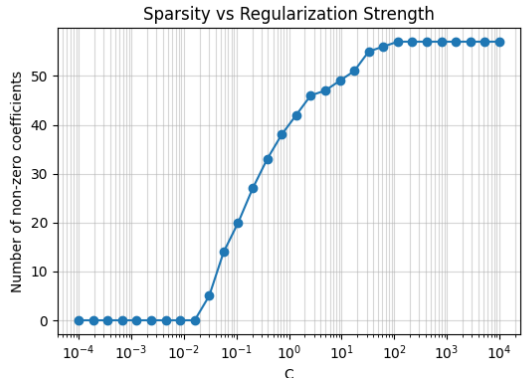


(b) Model loss with varying λ.

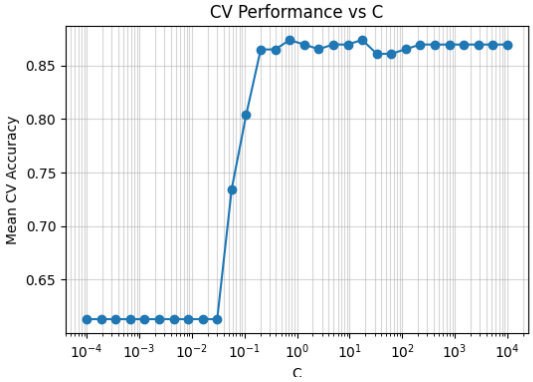
Fig. A.2: Accuracy and Loss minimization plots for Task 3



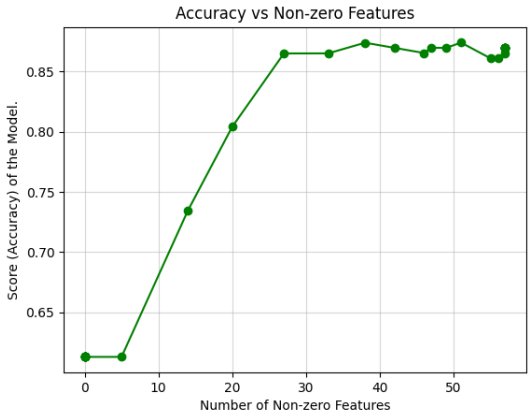
(a) L1 Regularization Path



(b) Sparsity vs. Regularization Strength



(c) Cross-Validation Performance vs. C



(d) Model Accuracy vs. C

Fig. A.3: Coefficient paths, sparsity, and performance metrics for Task 4

The following table shows the cross-validation results for all hyper-parameter configurations.

TABLE A.1: Full cross-validation results for all hyperparameter configurations.

eta	batch_size	epochs	mean_loss	std_loss	mean_acc	std_acc
0.0500	16	150	0.2203	0.0098	0.9001	0.0321
0.1000	32	150	0.2205	0.0100	0.9001	0.0321
0.0100	1	50	0.2209	0.0100	0.9001	0.0321
0.0500	16	200	0.2210	0.0082	0.9001	0.0321
1.0000	128	50	0.2211	0.0110	0.9001	0.0321
0.1000	64	200	0.2212	0.0113	0.9089	0.0380
0.1000	32	200	0.2213	0.0085	0.9001	0.0321
0.1000	16	100	0.2214	0.0084	0.9001	0.0321
0.1000	16	50	0.2237	0.0119	0.9089	0.0315
0.0500	16	100	0.2237	0.0115	0.9132	0.0322
0.1000	32	100	0.2238	0.0117	0.9132	0.0322
0.0500	32	200	0.2238	0.0117	0.9132	0.0322
0.1000	64	150	0.2248	0.0115	0.9132	0.0322
0.1000	16	150	0.2267	0.0083	0.8915	0.0369
0.1000	128	200	0.2269	0.0124	0.9131	0.0218
0.0200	16	200	0.2277	0.0121	0.9132	0.0322
1.0000	64	50	0.2279	0.0109	0.8958	0.0379
0.0100	1	100	0.2283	0.0096	0.8915	0.0369
0.0200	1	50	0.2290	0.0101	0.8915	0.0369
0.0500	32	150	0.2292	0.0123	0.9175	0.0264
0.1000	16	200	0.2331	0.0121	0.8915	0.0369
0.1000	128	150	0.2339	0.0118	0.9131	0.0218
0.1000	64	100	0.2344	0.0128	0.9131	0.0218
0.0010	1	200	0.2347	0.0128	0.9131	0.0218
0.0500	64	200	0.2348	0.0126	0.9131	0.0218
0.0200	16	150	0.2350	0.0126	0.9131	0.0218
1.0000	128	100	0.2352	0.0142	0.8958	0.0379
0.0100	1	150	0.2372	0.0151	0.8958	0.0315
0.1000	32	50	0.2406	0.0131	0.9131	0.0218
0.0500	16	50	0.2408	0.0132	0.9131	0.0218
0.0500	32	100	0.2409	0.0131	0.9131	0.0218
1.0000	128	150	0.2410	0.0172	0.9002	0.0369
1.0000	32	50	0.2415	0.0183	0.8958	0.0315
0.0100	1	200	0.2438	0.0190	0.8958	0.0315
0.0010	1	150	0.2441	0.0134	0.9131	0.0218
0.0500	64	150	0.2443	0.0128	0.9131	0.0218
1.0000	64	100	0.2444	0.0201	0.8915	0.0369
0.0200	1	100	0.2445	0.0196	0.8958	0.0315
1.0000	128	200	0.2474	0.0158	0.8959	0.0421
0.1000	128	100	0.2480	0.0134	0.9131	0.0218
0.0500	128	200	0.2486	0.0128	0.9131	0.0218
0.0100	16	200	0.2492	0.0133	0.9131	0.0218
0.0200	16	100	0.2493	0.0135	0.9131	0.0218
0.0200	32	200	0.2494	0.0134	0.9131	0.0218
1.0000	64	150	0.2499	0.0227	0.9045	0.0321
0.0500	1	50	0.2510	0.0228	0.9002	0.0369

0.0200	1	150	0.2521	0.0235	0.9002	0.0369
1.0000	64	200	0.2545	0.0213	0.9002	0.0369
1.0000	32	100	0.2547	0.0245	0.8959	0.0421
0.0200	1	200	0.2562	0.0246	0.8958	0.0315
1.0000	16	50	0.2581	0.0250	0.9002	0.0369
1.0000	32	150	0.2581	0.0253	0.8958	0.0315
0.0500	1	100	0.2602	0.0263	0.8915	0.0369
1.0000	32	200	0.2602	0.0245	0.8958	0.0315
0.1000	64	50	0.2607	0.0140	0.9087	0.0181
0.0500	128	150	0.2609	0.0129	0.9087	0.0181
0.0010	1	100	0.2610	0.0141	0.9087	0.0181
0.0500	64	100	0.2612	0.0140	0.9087	0.0181
0.0500	1	150	0.2618	0.0263	0.8958	0.0315
0.0200	32	150	0.2618	0.0139	0.9087	0.0181
0.0100	16	150	0.2619	0.0138	0.9087	0.0181
1.0000	16	100	0.2620	0.0262	0.8915	0.0369
1.0000	16	150	0.2621	0.0239	0.8915	0.0369
0.0500	1	200	0.2625	0.0256	0.8915	0.0369
1.0000	16	200	0.2627	0.0237	0.8958	0.0315
0.1000	1	50	0.2638	0.0263	0.8958	0.0315
0.1000	1	200	0.2651	0.0259	0.8958	0.0315
0.1000	1	150	0.2653	0.0262	0.8915	0.0369
0.1000	1	100	0.2659	0.0272	0.8958	0.0315
0.0500	32	50	0.2704	0.0143	0.9087	0.0181
0.0200	64	200	0.2725	0.0139	0.9087	0.0181
0.1000	128	50	0.2806	0.0148	0.9001	0.0304
0.0500	128	100	0.2814	0.0149	0.9044	0.0243
0.0200	32	100	0.2824	0.0148	0.9044	0.0243
0.0100	32	200	0.2825	0.0147	0.9044	0.0243
0.0200	16	50	0.2825	0.0149	0.9044	0.0243
0.0100	16	100	0.2826	0.0149	0.9044	0.0243
0.0200	64	150	0.2881	0.0146	0.9001	0.0304
0.0200	128	200	0.2946	0.0141	0.9001	0.0304
0.0010	1	50	0.2979	0.0153	0.9001	0.0304
0.0500	64	50	0.2983	0.0153	0.9001	0.0304
0.0100	32	150	0.2992	0.0152	0.9001	0.0304
0.0200	128	150	0.3128	0.0152	0.9001	0.0304
0.0200	64	100	0.3129	0.0157	0.9001	0.0304
0.0100	64	200	0.3131	0.0151	0.9001	0.0304
0.0500	128	50	0.3241	0.0159	0.9001	0.0304
0.0200	32	50	0.3254	0.0156	0.9001	0.0304
0.0100	32	100	0.3256	0.0157	0.9001	0.0304
0.0100	16	50	0.3257	0.0157	0.9001	0.0304
0.0100	64	150	0.3329	0.0156	0.9001	0.0304
0.0200	128	100	0.3410	0.0165	0.9001	0.0400
0.0100	128	200	0.3412	0.0153	0.9044	0.0339
0.0001	1	200	0.3625	0.0159	0.9044	0.0339
0.0200	64	50	0.3635	0.0159	0.9044	0.0339
0.0100	64	100	0.3639	0.0161	0.9044	0.0339
0.0100	128	150	0.3639	0.0160	0.9044	0.0339

0.0100	32	50	0.3790	0.0158	0.9044	0.0339
0.0001	1	150	0.3865	0.0161	0.9044	0.0339
0.0200	128	50	0.3977	0.0163	0.9044	0.0339
0.0100	128	100	0.3982	0.0168	0.9001	0.0400
0.0010	16	200	0.3987	0.0159	0.9044	0.0339
0.0001	1	100	0.4224	0.0168	0.9001	0.0304
0.0100	64	50	0.4240	0.0167	0.9001	0.0304
0.0010	16	150	0.4245	0.0166	0.9001	0.0304
0.0100	128	50	0.4605	0.0186	0.9001	0.0304
0.0010	32	200	0.4609	0.0185	0.9001	0.0304
0.0010	16	100	0.4609	0.0185	0.9001	0.0304
0.0001	1	50	0.4838	0.0206	0.9001	0.0304
0.0010	32	150	0.4856	0.0206	0.9001	0.0304
0.0010	64	200	0.5035	0.0224	0.8957	0.0279
0.0010	32	100	0.5171	0.0241	0.8957	0.0279
0.0010	16	50	0.5172	0.0241	0.8957	0.0279
0.0010	64	150	0.5245	0.0250	0.8957	0.0279
0.0010	128	200	0.5322	0.0259	0.9000	0.0219
0.0010	64	100	0.5492	0.0284	0.9000	0.0219
0.0010	128	150	0.5493	0.0284	0.9000	0.0219
0.0010	32	50	0.5585	0.0298	0.8957	0.0279
0.0010	128	100	0.5684	0.0313	0.9000	0.0219
0.0001	16	200	0.5685	0.0313	0.9000	0.0219
0.0010	64	50	0.5789	0.0329	0.8957	0.0181
0.0001	16	150	0.5790	0.0330	0.8957	0.0181
0.0001	32	200	0.5901	0.0349	0.8957	0.0181
0.0010	128	50	0.5901	0.0347	0.9000	0.0219
0.0001	16	100	0.5901	0.0349	0.8957	0.0181
0.0001	32	150	0.5959	0.0359	0.8957	0.0181
0.0001	64	200	0.5995	0.0365	0.9000	0.0219
0.0001	32	100	0.6020	0.0370	0.9000	0.0219
0.0001	16	50	0.6020	0.0370	0.9000	0.0219
0.0001	64	150	0.6032	0.0372	0.9000	0.0219
0.0001	128	200	0.6044	0.0374	0.9000	0.0219
0.0001	64	100	0.6069	0.0379	0.9000	0.0219
0.0001	128	150	0.6069	0.0379	0.9044	0.0161
0.0001	32	50	0.6082	0.0381	0.9044	0.0266
0.0001	128	100	0.6094	0.0383	0.9000	0.0219
0.0001	64	50	0.6107	0.0386	0.9000	0.0219
0.0001	128	50	0.6120	0.0388	0.8957	0.0181
1.0000	1	100	0.7623	0.3259	0.8959	0.0559
1.0000	1	50	0.7743	0.1417	0.8784	0.0610
1.0000	1	150	0.8110	0.3339	0.9000	0.0323
1.0000	1	200	1.0340	0.2703	0.9001	0.0217